

MANUAL TÉCNICO Y GUÍA DE INSTALACIÓN

Proyecto 2: Sonda de Kernel en C y Daemon en Go para la Telemetría de Contenedores

Universidad de San Carlos de Guatemala

Facultad de Ingeniería

Ingeniería en Ciencias y Sistemas

Sistemas Operativos 1

Carnet: 201700698

Nombre: Tulio Pirir

Fecha: Septiembre 2026

1. RESUMEN

Este proyecto implementa un sistema de monitoreo y gestión autónoma de contenedores Docker en Linux, compuesto por cuatro componentes principales:

1. Un módulo de kernel en C que expone métricas del sistema operativo a través del sistema de archivos virtual /proc.
2. Un daemon en Go que lee esas métricas, decide autónomamente qué contenedores eliminar cuando se violan restricciones de recursos, y persiste la información en una base de datos en memoria.
3. Un cronjob que cada minuto crea contenedores de prueba para simular carga variable.
4. Un dashboard en Grafana que visualiza en tiempo real las métricas recolectadas.

El sistema demuestra la integración entre programación a bajo nivel (kernel) y alto nivel (Go), abordando un problema real en entornos contenerizados: la monitorización y estabilización autónoma de recursos.

2. ARQUITECTURA DEL SISTEMA

El flujo de información es el siguiente:

Cronjob (cada 1 minuto)

↓ crea

Contenedores Docker (3 bajos + 2 altos + intruso ocasional)

↓ son leídos por

Módulo Kernel (expone /proc/continfo_pr2_so1_201700698)

↓ es leído por

Daemon Go (ciclo cada 30 segundos)

↓ escribe en

Valkey (base de datos en memoria)

↓ es leída por

Grafana (dashboard de visualización)

3. MÓDULO DE KERNEL

3.1 Ubicación

kernel_module/continfo.c

3.2 Descripción

Módulo de kernel cargable (LKM) que crea el archivo /proc/continfo_pr2_so1_201700698.

Cada lectura del archivo genera un JSON con:

- Memoria RAM total, libre y en uso (en MB)
- Timestamp Unix y fecha legible
- Lista de todos los procesos del sistema con:
 - PID del proceso
 - PPID (PID del padre)
 - Nombre del proceso
 - Línea de comandos completa
 - VSZ (tamaño de memoria virtual en KB)
 - RSS (memoria física residente en KB)
 - Porcentaje de memoria utilizada
 - Tiempo de CPU acumulado (en jiffies)

3.3 Estructuras del kernel utilizadas

Estructura	Uso
task_struct	Representa cada proceso en el kernel
mm_struct	Contiene información de memoria del proceso
sysinfo	Información global de memoria del sistema
timespec64	Manejo de timestamps

3.4 Funciones principales

Función	Descripción
continfo_init	Se ejecuta al cargar el módulo con insmod
continfo_show	Genera el contenido del archivo /proc
get_task_cmdline	Extrae la línea de comandos de un proceso
get_timestamp_string	Formatea la fecha y hora actual
continfo_exit	Se ejecuta al descargar el módulo con rmmod

3.5 Compilación

Requisitos:

- Headers del kernel (linux-headers-\$(uname -r))
- Build tools (build-essential)

Comandos:

```
cd kernel_module  
make
```

Resultado: archivo continfo.ko

3.6 Carga y descarga

```
# Cargar
sudo insmod continfo.ko

# Verificar
lsmod | grep continfo
ls -la /proc/continfo_pr2_so1_201700698

# Leer
cat /proc/continfo_pr2_so1_201700698 | head -30

# Descargar
sudo rmmod continfo
```

3.7 Verificación del log del kernel

```
sudo dmesg | tail -5
```

4. DAEMON EN GO

4.1 Ubicación

daemon/cmd/daemon/main.go

4.2 Descripción

Proceso en segundo plano que orquesta todo el sistema. Ciclo infinito cada 30 segundos que:

1. Lee el archivo /proc del módulo
2. Parsea el JSON
3. Extrae métricas de RAM y procesos
4. Escribe datos en Valkey con prefijo so1:
5. Evalúa contenedores Docker y mata los que exceden restricciones
6. Registra cada kill en Valkey

4.3 Lógica de decisión de contenedores

Restricciones estrictas:

Tipo	Cantidad mínima	Cantidad máxima
Contenedores bajo consumo	3	3
Contenedores alto consumo (RAM + CPU)	2	2
Contenedor intruso	0	1
Grafana	1	1 (nunca se toca)

Cuando hay exceso:

1. Se listan los contenedores por tipo (usando labels Docker)
2. Se obtiene el PID raíz de cada contenedor
3. Se calcula el RSS total del subárbol de procesos (contenedor + hijos)
4. Se ordenan de mayor a menor consumo
5. Se matan los que exceden el límite, empezando por los de mayor consumo

4.4 Claves escritas en Valkey

Clave	Tipo	Descripción
so1:ram:total_mb	String	RAM total en MB
so1:ram:free_mb	String	RAM libre en MB
so1:ram:used_mb	String	RAM en uso en MB
so1:ram:timestamp	String	Timestamp Unix
so1:ram	Hash	RAM completa (total, free, used, timestamp)
so1:ram:points	List	Histórico de puntos para gráfica
so1:processes:count	String	Total de procesos activos

so1:top:ram	Hash	Top 5 procesos por RSS
so1:top:cpu	Hash	Top 5 procesos por CPU
so1:kills:count	String	Total de contenedores eliminados
so1:kills:list	List	Historial de eliminaciones
so1:ebpf:events:count	String	Contador de eventos eBPF (pendiente)
so1:snapshot:latest	String	JSON completo del último snapshot

4.5 Compilación

Requisitos:

- Go 1.20+

```
cd daemon
go mod init so1/daemon
go get github.com/go-redis/redis/v8
go build -o bin/so1-daemon ./cmd/daemon
```

4.6 Ejecución

```
# Foreground (para ver logs)
./bin/so1-daemon

# Background
nohup ./bin/so1-daemon > /tmp/daemon.log 2>&1 &
```

4.7 Manejo de señales

El daemon captura SIGTERM y SIGINT. Al recibirlos:

1. Elimina el cronjob de crontab
2. Termina limpiamente

Esto cumple con el requisito del proyecto: "Antes de finalizar, el servicio deberá eliminar el cronjob asociado".

5. CRONJOB

5.1 Ubicación

scripts/create_containers.sh

5.2 Descripción

Script Bash que se ejecuta cada minuto. Verifica el estado de los contenedores y crea los faltantes.

5.3 Lógica

1. Cuenta contenedores activos usando labels (proyecto=so1, tipo=X)
2. Si hay menos de 3 bajos, crea los faltantes
3. Si hay menos de 2 altos (RAM + CPU), crea los faltantes
4. Con 20% de probabilidad y si no hay intrusos activos, crea uno
5. Registra cada acción en /tmp/cron_so1.log

5.4 Instalación

El daemon instala automáticamente el cronjob al arrancar. El crontab queda:

```
* * * * * /home/tj/workspaces/SistemasOperativos1/Proyecto2/scripts/create_containers.sh  
>> /tmp/cron_so1.log 2>&1
```

Los 5 asteriscos significan: cada minuto, cada hora, cada día, cada mes, cada día de la semana.

5.5 Logs

```
tail -20 /tmp/cron_so1.log
```

6. IMÁGENES DOCKER

6.1 Listado

Imagen	Dockerfile	Descripción
so1_bajo	docker_images/bajo_consumo/Dockerfile	Consumo mínimo (sleep infinity)
so1_alto_ram	docker_images/alto_ram/Dockerfile	Consumo ~512 MB RAM (stress-ng vm)
so1_alto_cpu	docker_images/alto_cpu/Dockerfile	4 workers de CPU al 100%
so1_intruso	docker_images/intruso/Dockerfile	Lee /etc/passwd en bucle

6.2 Construcción

```
cd docker_images

docker build --label proyecto=so1 --label tipo=bajo \
  -t so1_bajo:latest ./bajo_consumo

docker build --label proyecto=so1 --label tipo=alto_ram \
  -t so1_alto_ram:latest ./alto_ram

docker build --label proyecto=so1 --label tipo=alto_cpu \
  -t so1_alto_cpu:latest ./alto_cpu

docker build --label proyecto=so1 --label tipo=intruso \
  -t so1_intruso:latest ./intruso
```

6.3 Por qué usar labels

Los labels permiten filtrar contenedores de forma confiable incluso cuando Docker tiene bugs con `--filter` ancestro después de `--rm`. Cada contenedor tiene:

- `proyecto=so1` → identifica que es de este proyecto
- `tipo=bajo|alto_ram|alto_cpu|intruso` → categoría

7. VALKER (Base de datos)

7.1 Descripción

Valkey es un fork de código abierto de Redis 7.2.4. Compatible con el protocolo Redis, lo que permite usar clientes y herramientas de Redis.

7.2 Instalación

En este proyecto se usa el contenedor `valkey/valkey:7.2-alpine` escuchando en el puerto 6379.

```
docker run -d --name valkey_metrics -p 6379:6379 valkey/valkey:7.2-alpine
```

7.3 Verificación

```
redis-cli -h localhost -p 6379 PING
# Esperado: PONG

redis-cli -h localhost -p 6379 KEYS 'so1:*
```

8. GRAFANA

8.1 Instalación

```
docker run -d --name so1-grafana \
  --restart unless-stopped \
  -p 3000:3000 \
  -e GF_SECURITY_ADMIN_USER=admin \
  -e GF_SECURITY_ADMIN_PASSWORD=admin \
  -e GF_INSTALL_PLUGINS=redis-datasource \
  grafana/grafana:latest
```

8.2 Configuración del datasource

- Tipo: Redis (plugin redis-datasource)
- Address: valkey_metrics:6379
- Password: (vacío)

Nota: como Grafana corre en Docker y Valkey en otro contenedor, se conectan por nombre de contenedor. Para que funcione, ambos deben estar en la misma red Docker:

```
docker network connect proyecto2_monitoring_net so1-grafana
```

8.3 Dashboard

Ocho paneles configurados:

1. TOTAL RAM (Stat) — GET so1:ram:total_mb
2. FREE RAM (Stat) — GET so1:ram:free_mb
3. USED RAM (Stat) — GET so1:ram:used_mb
4. TOTAL CONTENEDORES ELIMINADOS (Stat) — GET so1:kills:count
5. GRÁFICA DE USO DE RAM EN EL TIEMPO (Time series) — LRANGE so1:ram:points 0 -1
6. TOP 5 +RAM (Pie chart) — HGETALL so1:top:ram
7. TOP 5 +CPU (Pie chart) — HGETALL so1:top:cpu
8. eBPF EVENT COUNTER (Stat) — GET so1:ebpf:events:count

Acceso: <http://localhost:3000> (admin/admin)

